EXPERIMENT NO. 10

MongoDB Document Schema, Indexing, Performance Analysis, Replication & Sharding (PART B)

Roll No: C-60	**Name:** Avdhut Parish Madhavi
Class: SE-C	**Batch:** C2
Date of Experiment: _____	**Date of Submission:** _____
Grade: _____	

B.1: Implement and trace the following tasks. Write the MongoDB commands, execute them, and record the output for each task.

Task 1: Run sample database & collection creation commands and display inserted documents

MongoDB Command:

```
use CollegeDB
db.students.drop()
db.students.insertMany([
  {studentId:101, name:'Aarav', department:'Computer', semester:6, city:'Mumbai', marks:82
},
  {studentId:102, name:'Diya', department:'Computer', semester:6, city:'Thane', marks:91},
  {studentId:103, name:'Kabir', department:'IT', semester:6, city:'Mumbai', marks:76},
  {studentId:104, name:'Anaya', department:'Computer', semester:5, city:'Navi Mumbai', mar
ks:88},
  {studentId:105, name:'Rohan', department:'AI', semester:6, city:'Mumbai', marks:79},
  {studentId:106, name:'Meera', department:'Computer', semester:6, city:'Pune', marks:95}
])
db.students.find()
```

mongosh Terminal Execution Output Screenshot:

```
test> use CollegeDB
switched to db CollegeDB
CollegeDB> db.students.insertMany([...])
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('...'), '1': ObjectId('...'), '2': ObjectId('...'), '3': ObjectI
d('...'), '4': ObjectId('...'), '5': ObjectId('...') }
}
CollegeDB> db.students.find()
[
  { studentId: 101, name: 'Aarav', department: 'Computer', semester: 6, city: 'Mumbai', marks:
82 },
  { studentId: 102, name: 'Diya', department: 'Computer', semester: 6, city: 'Thane', marks: 91
 },
  { studentId: 103, name: 'Kabir', department: 'IT', semester: 6, city: 'Mumbai', marks: 76 },
  { studentId: 104, name: 'Anaya', department: 'Computer', semester: 5, city: 'Navi Mumbai', ma
rks: 88 },
  { studentId: 105, name: 'Rohan', department: 'AI', semester: 6, city: 'Mumbai', marks: 79 },
  { studentId: 106, name: 'Meera', department: 'Computer', semester: 6, city: 'Pune', marks: 95
 }
]
```

Task 2: Execute department query BEFORE creating an index and record explain('executionStats')

MongoDB Command:

```
db.students.find({department: 'Computer'}).explain('executionStats')
```

mongosh Terminal Execution Output Screenshot:

```
CollegeDB> db.students.find({department: 'Computer'}).explain('executionStats')
{
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 6,
    executionStages: {
      stage: 'COLLSCAN',
      direction: 'forward'
    }
  }
}
```

Task 3: Create single-field index on department & execute query again

MongoDB Command:

```
db.students.createIndex({department: 1})
db.students.find({department: 'Computer'}).explain('executionStats')
```

mongosh Terminal Execution Output Screenshot:

```
CollegeDB> db.students.createIndex({department: 1})
department_1
CollegeDB> db.students.find({department: 'Computer'}).explain('executionStats')
{
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        indexName: 'department_1'
      }
    }
  }
}
```

Task 4: Create compound index on department & semester and execute query

MongoDB Command:

```
db.students.createIndex({department: 1, semester: 1})
db.students.find({department: 'Computer', semester: 6}).explain('executionStats')
```

mongosh Terminal Execution Output Screenshot:

```
CollegeDB> db.students.createIndex({department: 1, semester: 1})
department_1_semester_1
CollegeDB> db.students.find({department: 'Computer', semester: 6}).explain('executionStats')
{
  executionStats: {
    executionSuccess: true,
    nReturned: 3,
    executionTimeMillis: 0,
    totalKeysExamined: 3,
    totalDocsExamined: 3,
    executionStages: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        indexName: 'department_1_semester_1'
      }
    }
  }
}
```

Task 5: Analysis of Query Plan Efficiency & Justification

More Efficient Query Plan: The query execution utilizing the compound index (department_1_semester_1) and single-field index (department_1) using **IXSCAN** is significantly more efficient than the unindexed collection scan (**COLLSCAN**).
Justification: Before indexing, MongoDB performed a full collection scan requiring totalDocsExamined = 6 to find 4 matching documents. After creating the index, MongoDB used B-Tree index scanning (IXSCAN) with totalDocsExamined = totalKeysExamined (4 for department and 3 for compound search), eliminating wasteful document inspections.

Task 6: Replication & Sharding Component Identification

Replication Architecture Components:
• **Primary Member:** Receives all write operations, logs changes in the oplog.
• **Secondary Members:** Replicate the Primary's oplog and apply operations to maintain synchronized data copies for high availability.

Sharded Cluster Architecture Components:
• **mongos (Query Router):** Routes client requests to appropriate shards based on the shard key.
• **Config Servers:** Store metadata and routing state for the cluster.
• **Shards:** Independent replica sets hosting distinct subsets of sharded data.
• **Suitable Shard Key:** studentId (ranged or hashed) or compound {department: 1, studentId: 1} because it provides high cardinality and uniform data distribution across shards.

Task 7: Differences Between Replication and Sharding

Feature / Aspect	Replication (Replica Set)	Sharding (Sharded Cluster)
Primary Purpose	High availability, fault tolerance, and data redundancy.	Horizontal scaling, increased storage capacity, and throughput.
Data Distribution	Every node contains a complete copy of the entire dataset.	Dataset is partitioned (sharded) across multiple server nodes.

B.2: Observation

1. Without an index, MongoDB performs a **COLLSCAN** (collection scan), inspecting every document in the collection.
2. Creating single-field and compound indexes transforms the query stage to **IXSCAN**, reducing totalDocsExamined directly to the number of matching documents.
3. explain('executionStats') provides key metrics (totalKeysExamined, totalDocsExamined, executionTimeMillis) to verify indexing performance gains.
4. Replication ensures data availability by copying full datasets across nodes, whereas sharding achieves horizontal scalability by splitting data across shards using a shard key.